

Week 3 Lab — Control Flow & Functions

Decide · Repeat · Recover · Total: 35 marks (+3 bonus) · Due: end of Week 3

Name: _____ Student ID: _____ Program: _____

Instructions

- Work in a notebook named **Week3_YourName**. You need the dataset file **week3_students.csv** in the same folder (Colab: folder icon → upload).
- The dataset has **40 students** — and **3 records have broken scores** (values that are not numbers). Finding and surviving them is part of the lab.
- **Submit before the end of the week:** (1) completed notebook, (2) screenshots, (3) a ½–1 page report naming the 3 broken records and how your code survived them.
- Everything you need was covered in the Week 3 session — check the slides before searching the internet.

The Dataset — week3_students.csv

Columns (by index for `split(",")`): 0 student_id · 1 name · 2 age · 3 gender · 4 program · 5 district · 6 attendance_pct · 7 score

ID	Name	Age	G	Program	District	Att%	Score
AUCA001	Aline Uwase	19	F	Information Systems	Gasabo	92	85
AUCA002	Eric Niyonzima	22	M	Software Engineering	Musanze	88	78
AUCA003	Diane Mukamana	25	F	Networking	Muhanga	95	91
AUCA004	Jean Bosco Habimana	21	M	Accounting	Kicukiro	70	64
AUCA005	Sandrine Ingabire	24	F	Information Systems	Huye	85	72
AUCA006	Patrick Mugisha	20	M	Software Engineering	Nyagatare	90	83
...	... 34 more rows — 3 of them broken! ...						

Part 1 · Making Decisions (5 marks)

Exercise 1.1 — The grading ladder (3 marks)

Given a variable `score`, print the letter grade with an `if/elif/else` ladder: A (≥ 85) · B (70–84) · C (60–69) · D (50–59) · F (< 50). Test with 91, 76, 64, 52, 43.

Exercise 1.2 — Combined conditions (2 marks)

AUCA rule: a course is validated only if `score >= 50` **and** `attendance >= 75`. Print "Validated", "Blocked: low attendance" (passed but `attendance < 75`), or "Failed". Test with (76, 92), (76, 68), (43, 95).

Question 1.3 — Think about it (included in Part 1 marks)

What would go wrong if `elif score >= 50: came before elif score >= 70:`? Answer in one sentence.

Part 2 · Loops & the Accumulator (7 marks)

```
scores = [72, 85, 91, 64, 78, 47, 88, 55, 93, 61] # given for Part 2
```

Exercise 2.1 — Loop + filter (2 marks)

With a `for` loop, print only the scores ≥ 80 , each as: *Top score: X*

Exercise 2.2 — The accumulator pattern (3 marks)

Using **one loop** and NO built-ins (`sum`, `len`, `max` forbidden here): compute (a) total, (b) count, (c) average, (d) passed (≥ 50) and failed counts.

Initialize before the loop → update inside → report after.

Exercise 2.3 — while loop (2 marks)

You save 15,000 RWF per month toward a 250,000 RWF laptop. Use a while loop to compute how many months you need. Careful: avoid the infinite loop!

Part 3 · Functions (6 marks)**Exercise 3.1 — get_grade (3 marks)**

Turn your 1.1 ladder into `get_grade(score)` that **returns** the letter. Include a one-line docstring. Test on 91, 76, 43.

Exercise 3.2 — pass_rate (3 marks)

Write `pass_rate(score_list)` that returns the percentage of scores ≥ 50 , using an accumulator inside the function. On the Part 2 list, expected result: 90.0.

Part 4 · Reading the File (5 marks)**Exercise 4.1 — First contact (2 marks)**

Open `week3_students.csv` with `with open(...) as f:` and `readlines()`. Print (a) the number of lines, (b) the header line, (c) the first student's line. In a comment: why 41 lines, not 40?

Exercise 4.2 — Extract a column (3 marks)

Loop over `lines[1:]`; for each line use `.strip().split(",")` and append the name (index 1) to a list `names`. Print the count (40) and the first 5 names.

Part 5 · The Resilient Pipeline (12 marks)**Exercise 5.1 — Survive the mess (4 marks)**

Loop over all rows. try to convert the score (`parts[7]`) to int: on success add to `total` and `count valid`; on `ValueError` count `bad` and append the `student_id` to `bad_ids`. Print `valid`, `bad`, `bad_ids`, and the average of valid scores (1 decimal).

Checkpoint: you should find exactly 3 broken records.

Exercise 5.2 — Grades & the top student (4 marks)

Among valid rows only: (a) count passed (≥ 50) and failed; (b) track the highest score and its student's name with two accumulators (`best_score = -1`, `best_name = ""`). Print both, plus the top student's letter grade via your `get_grade`.

Exercise 5.3 — Stretch: report by district (4 marks)

Which district has the highest average score? Use two dictionary accumulators — `sums[d] = sums.get(d, 0) + s` and `counts[d] = counts.get(d, 0) + 1` where `d = parts[5]` — then print every district's average.

There are 8 districts, 5 students each (among the 40).

Bonus (+3) — Your first output file

Write a small report to `week3-report.txt` with `open(..., "w")` and `f.write(...)`: student count, valid/bad, average, top student. Then read the file back and print it.

Part 6 · Reflection (required, checked for completion)

1. Your pipeline skipped 3 broken records out of 40 (7.5%). At what percentage would you stop trusting the dataset — and what would you do instead of skipping?
2. Describe one thing from today's session that surprised or confused you.

✓ Submission checklist

- ☐ Notebook renamed `Week3_YourName`
- ☐ All exercises run without errors
- ☐ 3 broken records identified
- ☐ Reflection answered
- ☐ Screenshots
- ☐ Short report
- ☐ Submitted before end of Week 3

Marking scheme: Part 1 — 5 · Part 2 — 7 · Part 3 — 6 · Part 4 — 5 · Part 5 — 12 · Total 35 (+3 bonus).